

SIMPLY FPU

by **Raymond Filiatreault**
Copyright 2003

Chap. 13 Commented example

The example presented in this chapter is based on a simple dialog box to compute the monthly payments on a mortgage based on the following formula:

$$PMT = P \cdot R \cdot \frac{(1+R)^N}{(1+R)^N - 1}$$

where:

P = Mortgage Principle

R = Monthly Interest Rate (expressed in decimal)

N = Number of months

PMT = Computed monthly payments

The dialog box itself was designed with the "WYSIWYG" feature of the Symantec Resource Editor Rs32.exe provided with the MASM32 package in the rc sub-folder. The only manual modification to the generated script was to add the ES_NUMBER style to two of the edit controls, that style not being an option with Rs32. The resource script is reproduced at the end of this document.

Three separate edit controls are provided for the user to input the mortgage principal, the annual interest rate (expressed as a %) and the number of years. Two radio buttons are also provided to choose between two procedures for computing monthly payments.

One procedure is based on the generally allowed practice in the USA of compounding interest on a monthly basis. For example, a 12% stated annual rate becomes a 1% monthly rate which, when compounded, is equivalent to a true 12.6825% annual rate.

The other procedure is based on a restriction imposed on Canadian chartered banks for mortgage loans where the stated annual rate cannot be compounded more than twice per year. For example, a 12% stated annual rate becomes a 6% semi-annual rate which, when compounded, is equivalent to a true 12.36% annual rate. The monthly rate must thus be computed according to the following formula where R_{SA} is the semi-annual rate:

$$R = (1 + R_{SA})^{1/6} - 1$$

When this computed monthly rate is compounded on a monthly basis, it would be equivalent to the semi-annual rate.

In order to simplify the program as much as possible, some limits have been imposed on the user input but may not even be noticed.

-- The input for the mortgage principal is restricted to numbers only (with the ES_NUMBER style for its edit control) and limited to a maximum of 9 characters (with the EM_SETLIMITTEXT message during the WM_INITDIALOG phase). This still allows for a mortgage input of up to 1 billion dollars which should generally be sufficient, but disallows the inclusion of pennies (which would be very rarely specified anyway) as part of the input. The main advantage of those restrictions is that it guarantees a positive binary 32-bit integer can be retrieved directly from the edit control without any need to parse the input for invalid characters and perform the conversion from ASCII with additional code.

-- The input for the number of years is also restricted to numbers only and limited to a maximum of 2 characters. This still allows for a mortgage life of up to 99 years, but disallows partial years (which is also

rarely specified) as part of the input. Advantages are the same as above.

-- The input for the annual rate is restricted to 9 characters (rates are rarely specified with more than 3 decimals). This, however, guarantees that the value of the numerical digits excluding the decimal delimiter would not exceed the maximum possible value of a positive 32-bit integer.

While parsing the annual rate, a few more restrictions generate an error message:

-- An annual rate exceeding 100% (which would be considered illegal loan-sharking). This guarantees that the $\log_2$ of any $(1+R)$ term will always be less than 1.

-- A "-" sign. The rate must be positive.

A lack of input or an input equal to 0 in any of the three edit controls also generates an error message and no computation is performed.

With pre-validated data, the computation can thus proceed without any risk of error. Some code has nevertheless been added before displaying the result to ascertain that no major problem has been encountered due to unforeseeable circumstances.

The code has been kept as simple as possible, without any attempt to optimize it for speed or size (speed optimization would be a waste of time and effort for such an application). It is also specific for the application. Even the code to convert the annual rate from a string to a floating point should not be generalized without modifying the parts which rely on the designed purpose and restrictions.

The provided code is fully tested. It can be assembled without modification with MASM32 if copied into a file with the .asm extension. If the resource script is copied into a file named rsrc.rc and placed in the same directory as the .asm file, the .exe file can be generated in a single step with the Project->Build All menu option of the QEditor in MASM32. Modifications to the code and/or assembly procedures will be required with other assemblers and/or IDEs.

The FPU instructions used with this example application are (in alphabetical order):

<u>F2XM1</u>	2 to the X power minus 1
<u>FADD</u>	Add two floating point values
<u>FBSTP</u>	Store BCD data to memory
<u>FCHS</u>	Change the sign of ST(0)
<u>FDIV</u>	Divide two floating point values
<u>FIDIV</u>	Divide ST(0) by an Integer located in memory
<u>FILD</u>	Load integer from memory
<u>FIMUL</u>	Multiply ST(0) by an Integer located in memory
<u>FINIT</u>	Initialize the FPU
<u>FLD</u>	Load real number
<u>FLD1</u>	Load the value of 1
<u>FMUL</u>	Multiply two floating point values
<u>FRNDINT</u>	Round ST(0) to an integer
<u>FSCALE</u>	Scale ST(0) by ST(1)
<u>FSTP</u>	Store real number and pop ST(0)
<u>FSTSW</u>	Store status word
<u>FSUB</u>	Subtract two floating point values
<u>FWAIT</u>	Wait while FPU is busy
<u>FXCH</u>	Exchange the top data register with another data register
<u>FYL2XP1</u>	$Y * \log_2(X+1)$

```
; #####
;
;               Mortgage payment calculator
;       Written with MASM32 by Raymond Filiatreault
;               August 2003
;
; #####
```

```

.386                                ; minimum processor needed for 32 bit
.model flat, stdcall                ; FLAT memory model & STDCALL calling
option casemap :none                ; set code to case sensitive

; #####

include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc
include \masm32\include\comctl32.inc

includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib
includelib \masm32\lib\comctl32.lib

; #####

return MACRO arg
    mov eax, arg
    ret
ENDM

WndProc PROTO :DWORD, :DWORD, :DWORD, :DWORD

; #####

PRINCIPLE    EQU    711
RATEPCT      EQU    712
YEARS        EQU    713
PAYMENT      EQU    714
COMPUTE      EQU    720
QUITS        EQU    721
USABUT       EQU    730
CANADABUT    EQU    731
AMERICAN     EQU    0

.data
    hDlg      dd     0
    hInstance dd     0

    mortgage  dd     0
    months    dd     0

    factor6    dd     6
    factor10   dd    10
    factor12   dd    12

    bcdtemp    dt     0

    radiobutton db     0           ; 0=USA, 1=Canada

    badinput   db     "Input error",0
    princerr   db     "Unacceptable input for principle",0
    raterr     db     "Unacceptable input for rate",0
    yearerr    db     "Unacceptable input for years",0
    invalid    db     "Invalid FPU operation detected",0

    buffer1    db     16 dup(0)

; #####

.code

start:

    invoke GetModuleHandle, NULL
    mov     hInstance, eax

```

```

    invoke InitCommonControls

    invoke DialogBoxParam,hInstance,700,NULL,ADDR WndProc,NULL

    invoke ExitProcess,eax

; #####

WndProc proc hWin    :DWORD,
               uMsg    :DWORD,
               wParam  :DWORD,
               lParam  :DWORD

    .if uMsg == WM_INITDIALOG
        push hWin
        pop  hDlg          ;save handle in a global variable
                           ;this avoids having to pass it as a
                           ;parameter whenever needed outside this proc

        invoke SendDlgItemMessage,hDlg,PRINCIPLE,EM_SETLIMITTEXT,9,0
        invoke SendDlgItemMessage,hDlg,RATEPCT,EM_SETLIMITTEXT,9,0
        invoke SendDlgItemMessage,hDlg,YEARS,EM_SETLIMITTEXT,2,0
        invoke CheckRadioButton,hDlg,USABUT,CANADABUT,USABUT
        return TRUE

    .elseif uMsg == WM_COMMAND
        mov     eax,wParam
        and     eax,0ffffh

        .if     eax == QUITS                ;Exit button clicked
            invoke EndDialog,hDlg,0

        .elseif eax == COMPUTE              ;Compute button clicked
            call  maincalc                  ;process input and display result
            return TRUE

        .elseif eax == USABUT              ;USA radiobutton clicked
            mov   radiobutton,0             ;store info
            return TRUE

        .elseif eax == CANADABUT           ;Canada radiobutton clicked
            mov   radiobutton,1             ;store info
            return TRUE
        .endif

    .elseif uMsg == WM_CLOSE
        invoke EndDialog,hDlg,0

    .endif

    return FALSE                ;use Windows defaults to handle other messages

WndProc endp

; #####

maincalc:

;*****
;***** Retrieve THE MORTGAGE PRINCIPAL input *****
;*****

    invoke GetDlgItemInt,hDlg,PRINCIPLE,0,0    ;retrieve as an integer
    mov   mortgage,eax                        ;store it
    or     eax,eax                            ;check if input > 0
    ja     @F                                ;jump if OK
    lea    eax,princerr                       ;error message for input of principal

inputerror:

```

```

    invoke  MessageBox,0,eax,ADDR badinput,MB_OK ;display error message
    ret

;*****
;***** Retrieve the NUMBER OF YEARS input *****
;*****

@@:
    invoke  GetDlgItemInt,hDlg,YEARS,0,0          ;retrieve as an integer
    mul     factor12                             ;convert it to months
    mov     months,eax                           ;store it
    or      eax,eax                             ;check if input > 0
    ja      @F                                   ;jump if OK
    lea     eax,yearerr                          ;error message for input of years
    jmp     inputerror

;*****
;***** Retrieve the ANNUAL RATE input *****
;*****

@@:
    invoke  GetDlgItemText,hDlg,RATEPCT,ADDR buffer1,16 ;retrieve as string
    .if     buffer1 == 0                        ;buffer1 being a global BYTE variable
                                                ;if there was no input in the EDIT control,
                                                ;the first byte of buffer1 would be 0

badrate:
    lea     eax,raterr                          ;error message for annual rate input
    jmp     inputerror
    .endif

    call    atofl                               ;convert ASCII string (%) to REAL10 decimal
                                                ;->st(0)=decimal annual rate (if no error)

    ; Note: The "atofl" sub-routine is called only once
    ; and its code could have been inserted directly
    ; without the need for the "call" instruction. This
    ; was done for the purpose of clarity in this section.

    or      eax,eax                             ;check if error was detected
    jz      badrate                             ;abort and display message if error

;*****
;***** Convert the annual rate to a MONTHLY rate *****
;*****

    .if     radiobutton == AMERICAN
        fidiv factor12                         ;divide the annual rate by 12
                                                ;-> st(0)=monthly rate
    .else
        fld1                                  ;-> st(0)=1, st(1)=annual rate
        fchs                                  ;-> st(0)=-1, st(1)=annual rate
        fxch                                  ;-> st(0)=annual rate, st(1)=-1 scaling factor
        fscale                                ;divide the annual rate by 2^1
                                                ;to get the semi-annual rate
                                                ;(a positive scaling factor will multiply
                                                ;a negative scaling factor will divide)
        fstp st(1)                            ;overwrite the scaling factor with st(0)
                                                ;and pop the top register
                                                ;-> st(0)=semi-annual rate
        fld1                                  ;-> st(0)=1, st(1)=semi-annual rate
        fidiv factor6                         ;-> st(0)=1/6, st(1)=semi-annual rate
        fxch                                  ;-> st(0)=semi-annual rate, st(1)=1/6
        fyl2xp1                               ;-> st(0)=[log2(semi-annual rate+1)]*1/6

        ;Note: Because of limitations imposed on input for size and sign,
        ;the (semi-annual rate+1) term will be between 1 and 1.5
        ;and its log2 will be positive and less than 1. That log2

```

```

;is further divided by 6 for a value definitely between
;0 and +1. That value can thus be used directly with
;the next instruction without any need for scaling before
;or after.

```

```

f2xm1      ;-> st(0)=monthly rate (obtained directly
           ;because the instruction provides the minus 1)
           ;When this monthly rate is compounded 6 times
           ; it would be equal to the semi-annual rate

```

```

.endif

```

```

;*****
;*****      Compute the monthly payments      *****
;*****

```

```

fild months      ;-> st(0)=months, st(1)=monthly rate (R)
fld st(1)         ;-> st(0)=R, st(1)=months, st(2)=R
fyl2xpl          ;-> st(0)=log2(1+R)*months, st(1)=R
fld st           ;-> st(0)=log, st(1)=log, st(2)=R
frndint          ;-> st(0)=int(log), st(1)=log, st(2)=R
fsub st(1),st     ;-> st(0)=int(log), st(1)=log-int(log), st(2)=R
fxch st(1)        ;-> st(0)=log-int(log), st(1)=int(log), st(2)=R
f2xm1            ;-> st(0)=2[log-int(log)]-1, st(1)=int(log), st(2)=R
fld1
fadd             ;-> st(0)=2[log-int(log)], st(1)=int(log), st(2)=R
fscale           ;-> st(0)=(1+R)N, st(1)=int(log), st(2)=R
fstp st(1)       ;-> st(0)=(1+R)N, st(1)=R
fld st          ;-> st(0)=(1+R)N, st(1)=(1+R)N, st(2)=R
fld1            ;-> st(0)=1, st(1)=(1+R)N, st(2)=(1+R)N, st(3)=R
fsub            ;-> st(0)=(1+R)N-1, st(1)=(1+R)N, st(2)=R
fdiv            ;-> st(0)=(1+R)N/[(1+R)N-1], st(1)=R
fmul            ;-> st(0)=R*(1+R)N/[(1+R)N-1]
fimul mortgage   ;-> st(0)=P*R*(1+R)N/[(1+R)N-1]=Monthly payments
fimul factor10   ;multiply by 100 to have 2 decimal places as integer
fimul factor10   ;-> st(0)=Monthly payments*100
fbstp bcdtemp    ;store in memory in BCD format
                ;rounded to the closest penny

```

```

;*****
; The following section of code is not necessary for this application
; because every precaution was taken to examine the input to insure that
; the data used in the FPU computations is valid and would not result in
; any major error. It is merely included to indicate how to check the
; validity of the end result whenever there may be a risk of invalid data.
; It also insures that the FBSTP instruction is completed before starting
; to access the stored packed BCD data.
;*****

```

```

fstsw ax         ;copy to AX the FPU's Status Word
                ;containing the exception flags
fwait           ;insure the execution is completed
shr eax,1        ;transfer bit0 to the CPU's carry flag
                ;That bit would be set if an invalid operation
                ;was detected with any of the FPU instructions
                ;The final result would then be invalid
jnc @F           ;continue if no invalid operation flag

lea eax,invalid
jmp inputerror   ;display error message and return to WndProc

```

```

;*****
;*****      Unpack the BCD result and display it      *****
;
; The coding used is not suitable as a general purpose unpacking
; algorithm for at least two reasons: the sign is known to be
; positive and thus disregarded, and the result was designed to
; always contain 2 decimal places.

```

```
;*****
```

```
@@:
    push esi          ;preserve ESI and EDI
    push edi
    lea esi,bcdtemp+8 ;use ESI to point initially
                        ;to the 2nd most significant byte
    lea edi,buffer1   ;use EDI to point to the buffer
                        ;where the ASCII string will be stored
    mov ecx,8         ;use ECX as counter for the number of bytes
                        ;possibly containing integer digits
```

```
    ;Note: (The BCD format has 10 bytes. The most significant
            ;byte contains the sign which is disregarded in this
            ;application, and the least significant byte is known
            ;to contain two decimal digits.)
```

```
;*****
; Search for the most significant byte containing the 1st integer digit
;*****
```

```
@@:
    movzx eax,byte ptr[esi] ;get next byte in AL zero extended in EAX
    dec esi                ;adjust pointer to next byte
    or  eax,eax            ;check if it contains the 1st integer
    jnz @F                 ;jump if found
    dec ecx                ;decrement counter
    jnz @B                 ;continue search until all integer bytes checked
```

```
;*****
; If no integer digit is present,
; place a "0" digit before the decimal delimiter
;*****
```

```
    mov al,"0"
    stosb                ;insert the "0" digit
    jmp decimals         ;go insert the decimal delimiter and digits
```

```
;*****
; If that 1st byte contains only 1 integer digit,
; it has to be processed separately to avoid a leading 0
;*****
```

```
@@:
    test al,0f0h         ;check the high nibble of AL
    jnz @F               ;jump if there are 2 integer digits
    add al,"0"           ;convert the digit to ASCII
    stosb                ;insert that digit
    jmp nextdigit        ;process next byte
```

```
;*****
; Other bytes contain 2 integer digits each which must be unpacked
; The high nibble digit must be followed in memory by the low nibble
;*****
```

```
@@:
    ror ax,4             ;transfer the high nibble to low nibble of AL
                        ;and low nibble of AL to high nibble of AH
    ror ah,4             ;transfer it to the low nibble of AH
    add ax,3030h         ;convert both to ASCII
    stosw                ;insert both, AL followed by AH in memory
```

```
nextdigit:
    movzx eax,byte ptr[esi] ;get next byte
    dec esi                ;adjust pointer
    dec ecx                ;decrement integer byte counter
    jnz @B                 ;continue with integer digits until completed
```

```
decimals:
```



```

    mov     byte ptr[edi], "." ;insert the decimal delimiter
    inc     edi                ;adjust string pointer
    ror     ax,4               ;unpack the decimal byte as above
    ror     ah,4
    add     ax,3030h
    stosw
    mov     byte ptr[edi],0    ;insert the terminating 0

    pop     edi                ;restore the EDI and ESI registers
    pop     esi

;*****
; Display the result in the dialog box and return control to the WndProc
;*****

    invoke SetDlgItemText,hDlg,PAYMENT,ADDR buffer1
    ret

;*****
;
;          atofl
;
; This sub-routine is partly general purpose and partly specific
; for the task. It parses the string for unacceptable characters
; but also returns an error if the integral portion of the input
; exceeds 100. It also returns an error for a negative sign.
;
; The conversion approach of treating all the numerical digits as
; being integer digits is sound only because the size of the
; string was limited to 9 characters in the EDIT control. This
; guarantees that, in a worse case scenario, the maximum integer
; value of the input would still fit in a 32-bit register. The
; final integer value obtained is then corrected according to the
; count of decimal digits in the input string.
;
; Returns with EAX = 0 if error detected.
;
; If EAX != 0, the converted annual rate is returned in st(0)
; already divided by 100.
;*****

atofl:
    push    ebx               ;preserve EBX and ESI
    push    esi

    lea     esi,buffer1 ;use ESI as pointer to text buffer
    xor     eax,eax
    xor     ebx,ebx          ;will be used as an accumulator
    xor     ecx,ecx          ;will be used as a counter

;*****
; Skip leading spaces without generating an error
;*****

@@:
    lodsb                ;get next character
    cmp     al," "        ;check if a space character
    jz      @B            ;repeat until a non-space character is found

;*****
; Check 1st non-space character for a +/- sign
;*****

    cmp     al,"-"        ;is it a "-" sign
    jnz     @F

atoflerr:
    xor     eax,eax        ;set EAX to error code
    pop     esi            ;restore the EBX and ESI registers
    pop     ebx

```



```

ret                                ;return with error code

@@:
  cmp    al,"+"                    ;is it a "+" sign
  jnz    nextchar                  ;disregard a "+" sign and get next character
  lodsb                                     ;disregard a "+" sign and get next character

;*****
; From this point, space and sign characters will be invalid
;*****

nextchar:
  cmp    al,0                      ;check for end-of-string character
  jz     endinput                  ;exit the string parsing section

  cmp    al,"."                    ;is it the "." decimal delimiter
                                   ;other delimiters such as the "," used in some
                                   ;countries could also be allowed but would need
                                   ;additional coding to make it more generalized
  jnz    @F

;*****
; Only one decimal delimiter can be acceptable. The sign bit of ECX
; is used to keep a record of the first delimiter identified.
;*****

  or     ecx,ecx                    ;check if a delimiter has already been identified
  js     atoflerr                  ;exit with error code if more than 1 delimiter

  stc                                     ;set the carry flag
  rcr    ecx,1                     ;set bit31 of ECX (the sign bit) when
                                   ;the 1st delimiter is identified
  lodsb                                ;get next character
  jmp    nextchar                  ;continue parsing

;*****
; All ASCII characters other than the numerical ones will now be invalid
;*****

@@:
  cmp    al,"0"
  jb     atoflerr
  cmp    al,"9"
  ja     atoflerr

  sub    al,"0"                    ;convert valid ASCII numerical character to binary
  xchg   eax,ebx                   ;get the accumulated integer value in EAX
                                   ;holding the new digit in EBX
  mul    factor10                  ;multiply the accumulated value by 10
  add    eax,ebx                   ; and add the new digit
  xchg   eax,ebx                   ;store this new accumulated value back in EBX

  or     ecx,ecx                    ;check if a decimal delimiter detected yet
  js     @F                        ;jump if decimal digits are being processed

;*****
; Integer digits still being processed
;*****

  cmp    ebx,100                   ;verify current value of integer portion
  ja     atoflerr                  ;abort if input for annual rate is > 100%
  lodsb                                ;get next string character
  jmp    nextchar                  ;continue processing string characters

;*****
; The CL register is used as a counter of decimal digits
; after the decimal delimiter has been identified
;*****

```

```

@@:
    inc    cl            ;increment count of decimal digits
    lodsb           ;get next string character
    jmp    nextchar      ;continue processing string characters

;*****
; Parsing of the string is completed
;*****

endinput:
    or     ebx,ebx       ;check if total input was equal to 0
    jz     atoflerr      ;abort if annual rate input is 0%

    finit          ;initialize FPU
    push   ebx           ;store value of EBX on stack
    fild   dword ptr[esp] ;-> st(0)=EBX
    add    cl,2          ;increment the number of decimal digits
                        ;to convert from % rate to a decimal rate
    shl    ecx,1         ;get rid of the potential sign "flag"
    shr    ecx,1         ;restore the count of decimal digits
    fild   factor10      ;-> st(0)=10, st(1)=EBX
@@:
    fdiv   st(1),st       ;-> st(0)=10, st(1)=EBX/10
    dec    ecx           ;decrement counter of decimal digits
    jnz    @B            ;continue dividing by 10 until count exhausted
    fstp   st            ;get rid of the dividing 10 in st(0)
                        ;-> st(0)=annual rate (as a decimal rate)
    pop    ebx           ;clean CPU stack

    pop    esi           ;restore the EBX and ESI registers
    pop    ebx
    or     al,1          ;insure EAX != 0 (i.e. no error detected)
    ret

```

```

;*****

```

```

end start

```

```

;#####
;
;                               Resource script for the Dialog Box
;                               to be used with the Mortgage Calculator
;                               August 2003
;#####

```

```

#include "\masm32\include\resource.h"

```

```

700 DIALOGEX MOVEABLE IMPURE LOADONCALL DISCARDABLE    100, 80, 131, 132, 0
STYLE DS_MODALFRAME | 0x0004 | WS_CAPTION | WS_SYSMENU | WS_VISIBLE | WS_POPUP
CAPTION "Mortgage Payments"
FONT 8, "MS Sans Serif", 700, 0 /*FALSE*/
BEGIN
    LTEXT        "Mortgage principle", 701, 4,6,66,10, SS_LEFT, , 0
    LTEXT        "Annual rate, %", 702,    4,22,66,10, SS_LEFT, , 0
    LTEXT        "Number of years", 703,    4,38,66,10, SS_LEFT, , 0
    LTEXT        "Monthly payments", 704,    4,84,66,10, SS_LEFT, , 0
    EDITTEXT     711, 78, 6,49,11, ES_RIGHT | ES_NUMBER, , 0
    EDITTEXT     712, 78,22,49,11, ES_RIGHT, , 0
    EDITTEXT     713, 102,38,25,11, ES_RIGHT | ES_NUMBER, , 0
    CONTROL      "", 714, "Edit", ES_READONLY | ES_RIGHT, 78,84,49,11, , 0
    CONTROL      "Compute", 720, "Button", BS_DEFPUSHBUTTON, 16,111,41,13, , 0
    CONTROL      "Exit", 721, "Button", 0,    74,111,41,13, , 0
    CONTROL      "U.S.A.", 730, "Button", BS_AUTORADIOBUTTON | WS_GROUP, 18,62,44,10, , 0
    CONTROL      "Canada", 731, "Button", BS_AUTORADIOBUTTON, 75,62,44,10, , 0
END

```

[RETURN TO
SIMPLY FPU
CONTENTS](#)